

프롬프트에서 이미지로

학습 체크리스트 (1/2)

☐ 확산 모델과 잠재 확산의 동작 원리 이해

☐ `steps` · `guidance` · `seed`가 결과에 미치는 영향 파악

☐ Cloudflare Workers AI의 호출 구조와 준비물 습득

☐ 장면 묘사형 이미지 프롬프트 작성법과 LLM 보강 방법 습득

학습 체크리스트 (2/2)

☐ Workers AI가 OpenAI 호환 이미지 생성을 지원하지 않는 이유 이해

☐ `RestClient`로 커스텀 `ImageModel`을 구현하는 방법 습득

☐ Base64 응답을 Data URI로 즉시 표시하는 흐름 파악

☐ 선택적 저장과 seed 기반 재현성 확보 전략 이해

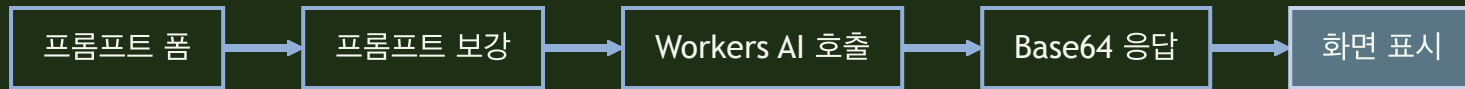
방향이 반대다

- 이미지 분석: 이미지를 입력으로 넣어 텍스트로 해석
- 이미지 생성: 텍스트를 입력으로 넣어 이미지를 생성
- 공통점: 둘 다 Spring AI로 멀티모달 모델 호출
- 차이점: 입력과 출력의 모달리티가 서로 반대

이번 차시에서 만들 것

- 입력: Thymeleaf 프롬프트 입력 폼
- 호출: Spring Boot가 Cloudflare Workers AI 호출
- 표시: 응답받은 Base64 이미지를 화면에 즉시 렌더링
- 저장: 저장 선택 시에만 객체 스토리지에 영구 저장

전체 흐름



미리보기와 저장을 한 요청에서

- 세션 부담: 거대한 Base64를 세션에 담기엔 부담이 큼
- 재전송 부담: hidden field·두 번째 요청 전달도 비효율적
- 선택: 생성과 저장을 한 요청 안에서 함께 처리
- 결과: 사용자는 제출 전에 저장 여부를 미리 선택

확산 모델이 이미지를 만드는 방식

확산 모델 (Diffusion Model)

무작위 노이즈에서 출발해 점진적으로 이미지를 복원하는 생성 모델

- **역방향 과정 (생성):** 순수한 가우시안 노이즈에서 시작해 노이즈를 조금씩 차감
- **순방향 과정 (학습):** 원본 이미지에 노이즈를 단계적으로 섞어 완전한 노이즈로 변환
- **학습 목표:** 신경망이 각 단계별로 "얼마만큼의 노이즈가 추가되었는가"를 예측하도록 학습
- **결과물:** 무작위 노이즈 패턴에서 시작하지만 매번 정밀한 이미지 구조를 완성

노이즈를 걷어내는 디노이징 과정

디노이징 (Denoising)

무작위 노이즈에서 신경망이 예측한 노이즈 성분을 스텝별로 걷어내어 선명한 형상을 복원하는 과정

- **스텝(Step) 단위 복원:** 한 번에 복원하지 않고 $t = T$ 부터 $t = 0$ 까지 수십 번 분할 수행
- **노이즈 예측 신경망:** U-Net / DiT 모델이 현재 입력 상태의 노이즈 패턴을 계산
- **점진적 형상 출현:** 초기 스텝에서는 거친 윤곽·구도가, 후반 스텝에서는 세부 질감이 완성
- **조건부 제어:** 프롬프트 임베딩 조건에 부합하는 방향으로 노이즈 걷어내기 방향을 유도

압축된 공간에서 연산하는 잠재 확산

잠재 확산 모델 (Latent Diffusion Model)

픽셀 공간이 아닌 압축된 잠재 공간에서 디노이징을 수행하는 확산 모델

- 픽셀 공간의 한계: 512×512 픽셀 연산은 GPU 메모리와 시간 소모가 큼
- 잠재 공간 전환: 이미지의 핵심 특징만 저차원 잠재 공간(Latent Space)에 응축
- 효율적 디노이징: $1/64$ 로 줄어든 공간에서 반복 연산을 수행해 속도 향상
- 최종 복원: 노이즈 제거 완료 후 디코더를 거쳐 고해상도 이미지로 변환

이미지를 압축하는 VAE 인코더

변분 오토인코더 (VAE, Variational Autoencoder)

고차원 픽셀 데이터를 의미 있는 저차원 잠재 표현으로 압축하고 복원하는 모델

- **특징 추출:** 단순 픽셀 나열 대신 **구도·색상·질감** 등 의미적 핵심 특징을 추출
- **차원 압축:** VAE 인코더가 $512 \times 512 \times 3$ 픽셀을 $64 \times 64 \times 4$ 잠재 표현으로 압축
- **연산 부담 절감:** 사람이 인지하기 어려운 픽셀 디테일을 압축해 디노이징 연산 대폭 절약
- **역할 분담:** VAE 인코더는 잠재 공간 압축을, VAE 디코더는 최종 픽셀 복원을 담당

텍스트가 생성을 조건화하는 방식

- 인코딩: 텍스트 인코더가 프롬프트를 임베딩으로 변환
- 개입: 디노이징 각 스텝에서 cross-attention으로 반영
- 텍스트 임베딩과 비교: 그때는 임베딩을 검색에 사용
- 이번 차시: 같은 임베딩을 생성의 조건으로 사용

생성 결과를 좌우하는 핵심 파라미터

파라미터	의미	영향
<code>steps</code>	디노이징 반복 횟수	많을수록 품질 ↑ 지연 ↑
<code>guidance</code>	프롬프트 반영 강도	높으면 충실, 과하면 부자연
<code>seed</code>	노이즈 초기값	고정 시 결과 재현 가능
<code>width</code> / <code>height</code>	출력 해상도	클수록 연산량 급증

증류로 스텝을 줄인 few-step 모델

- 증류(Distillation): 교사 모델의 결과를 학생 모델이 압축 학습
- 효과: 20~50 스텝을 1~4 스텝으로 줄여도 유사한 품질
- 대표 모델: FLUX.1 schnell(독일어로 "빠른")
- 적합 환경: 지연이 짧아 서버리스·엣지 환경에 적합

같은 프롬프트도 매번 다른 이미지

- 원인: seed를 고정하지 않으면 노이즈 초기값이 매번 다름
- 결과: 동일 프롬프트라도 실행마다 다른 이미지 생성
- 주의: 이미지나 그 해시로 캐시 키를 만들 수 없음
- 대안: 프롬프트와 seed 조합을 중복 판정 기준으로 사용

아직 남은 한계

- 텍스트 렌더링: 이미지 안 글자를 정확히 그리기 어려움
- 취약 사례: 로고나 정확한 문구가 필요한 요구사항
- 비용 구조: 해상도가 커질수록 처리할 타일 수가 증가
- 결과: 스텝당 연산량도 늘어 비용이 가파르게 상승

Cloudflare Workers AI

워커스 AI (Workers AI)

전 세계 엣지 네트워크의 GPU 위에서 오픈소스 모델을 서버리스로 실행해주는 추론 플랫폼

- 서버리스 추론: GPU 프로비저닝 없이 HTTP 요청 한 번
- 모델 카탈로그: 텍스트 생성·임베딩·이미지 생성·음성 제공

왜 Workers AI인가

- REST 직접 호출: 이미지 생성 모델을 `RestClient`로 연동 가능
- 동일 플랫폼: 텍스트 임베딩에서 다룬 `bge-m3`와 같은 플랫폼
- 요금 정책: 변동이 잦으므로 공식 문서에서 최신 내용 확인

호출 엔드포인트

```
1 POST https://api.cloudflare.com/client/v4
2     /accounts/{ACCOUNT_ID}/ai/run/{MODEL_ID}
3
4 Authorization: Bearer {API_TOKEN}
```

호출 전 준비물

- Account ID: Cloudflare 대시보드에서 확인
- API Token: Workers AI 실행 권한만 부여한 전용 토큰
- 모델 카탈로그: 추가·폐기가 잦아 실습 직전 확인 필요

FLUX.1 [schnell] 사양

항목	내용
모델 ID	@cf/black-forest-labs/flux-1-schnell
요청 형식	JSON (Content-Type: application/json)
파라미터	prompt 1~2048자 필수, steps 기본 4(4~8), seed 선택
출력	Base64 JPEG ({ "result": { "image": " ... " } })

이미지 프롬프트는 지시문이 아니다

- LLM 프롬프트: 모델에게 일을 시키는 지시·대화
- 이미지 프롬프트: 완성된 한 장면을 그대로 묘사
- 명령형 지양: "그려줘" 대신 명사구를 나열
- 효과: 서술형 나열이 결과 품질을 끌어올림

프롬프트 구성 요소

요소	예시
주제 · 배경	붉은 우산을 든 노년의 남성 · 비 내리는 밤의 서울 골목
스타일	수채화, 필름 사진, 아이소메트릭 3D 렌더
구도 · 시점	로우앵글 클로즈업, 광각 풀샷
조명 · 디테일	역광의 따뜻한 황금빛, 얇은 피사계 심도

요소를 한 문장에 녹인 예

```
1  An elderly man holding a red umbrella  
2  in a rainy Seoul alley at night,  
3  watercolor style, low-angle close-up,  
4  warm backlighting, highly detailed.
```


프롬프트 작성 시 주의점

- 네거티브 미지원: FLUX에는 negative prompt 파라미터가 없음
- 긍정문 사용: 부정문 대신 원하는 것을 직접 서술
- 영어 우선: 한국어는 텍스트 인코더 편중으로 품질 저하
- 길이 배치: 2048자 제한 안에서 핵심을 앞쪽에 배치

LLM으로 프롬프트 보강하기

- 목적: 짧고 모호한 한국어를 구조화된 영어 묘사로 확장
- 재사용: 텍스트 생성의 `ChatClient` 와 `.entity( ... )` 구조화 출력
- 시스템 프롬프트: 주제·배경·스타일·조명·구도 포함, 명령문 금지
- 원본 보존: 의도가 뒤틀릴 수 있으므로 원본을 항상 함께 보관

보강 프롬프트 호출

```
1 EnhancedPrompt result = chatClient.prompt()  
2     .system(ENHANCE_SYSTEM_PROMPT)  
3     .user(userInput)  
4     .call()  
5     .entity(EnhancedPrompt.class);
```

같은 방식이 통하지 않는 이유

- OpenAI 호환 계층: `/chat/completions`, `/embeddings` 만 지원
- 이미지 생성 미지원: `/images/generations` 엔드포인트가 없음
- base-url 변경 무의미: OpenAI 스타터 `ImageModel` 연결 불가
- Spring AI 2.0 한계: OpenAI·Azure OpenAI·Stability AI만 구현 존재

세 가지 선택지

방식	장점	단점
커스텀 <code>ImageModel</code>	Spring AI 공통 타입 유지	요청·응답 매핑을 직접 작성
REST 직접 호출	가장 단순, 제약 없음	모델 교체 시 서비스 전체 수정
AI Gateway 경유	캐싱·로깅·비용 분석 확보	게이트웨이 설정이 추가로 필요

본 실습의 선택

- 커스텀 `ImageModel`: 공통 입·출력 타입은 그대로 유지
- 공급자 전용 옵션: seed·steps 때문에 `CloudflareImageOptions` 사용
- 교체 영향 범위: 공급자 교체 시 서비스의 옵션 생성부도 수정
- 규약 유지: 컨트롤러·화면의 반환 규약은 변경 없음

새 의존성 없이 조립하기

- **ImageModel**: Spring AI 코어 모듈의 기본 타입 재사용
- **RestClient**: Spring Boot 표준 동기 HTTP 클라이언트 사용
- **프롬프트 보강**: 기존 Google GenAI 스타터 그대로 재사용
- **추가 스타터 불필요**: 기존 조합만으로 커스텀 구현 완성

Workers AI 설정 값

```
1  app:
2    ai:
3      image-generation:
4        account-id: ${CF_ACCOUNT_ID}
5        api-token: ${CF_API_TOKEN}
6        base-url: https://api.cloudflare.com/client/v4
7        model: "@cf/black-forest-labs/flux-1-schnell"
8        steps: 4
9        timeout: 60s
```


설정 작성 시 주의점

- 자격 증명: 환경변수로 참조하고 파일에 값을 직접 넣지 않음
- 예약 문자 @ : @cf/ 로 시작하는 모델 ID는 따옴표 필수
- Duration 바인딩: timeout 은 60s · 2m 표기를 그대로 사용
- 모델 교체: 설정으로 분리해 코드 수정 없이 교체 가능

설정 바인딩 record

```
1  @ConfigurationProperties(prefix = "app.ai")
2  public record AiProperties(ImageGeneration imageGeneration) {
3
4      public record ImageGeneration(String accountId, String apiToken,
5          String baseUrl, String model, int steps, Duration timeout) {}
6  }
```

커스텀 ImageModel 구현 순서

- RestClient 빈: base URL · 인증 헤더 · 타임아웃 고정
- 요청·응답 record: Cloudflare의 `result` 래퍼와 `success` · `errors`
- ImageOptions 구현체: 모델 ID · steps · seed 보유
- call(ImagePrompt): REST 호출 후 Base64를 `Image`에 담아 반환

Workers AI 전용 RestClient 빈

```
1  @Bean
2  RestClient workersAiRestClient(RestClient.Builder builder, ImageGeneration props) {
3      var settings = ClientHttpRequestFactorySettings.defaults()
4          .withConnectTimeout(Duration.ofSeconds(10))
5          .withReadTimeout(props.timeout());
6      return builder.baseUrl(props.baseUrl())
7          .requestFactory(ClientHttpRequestFactoryBuilder.detect().build(settings))
8          .defaultHeader(AUTHORIZATION, "Bearer " + props.apiToken())
9          .build();
10 }
```

요청·응답 DTO

```
1 public record CloudflareImageRequest(String prompt, int steps, Long seed) {}
2
3 public record CloudflareImageResponse(
4     CloudflareImageResult result, boolean success,
5     List<CloudflareError> errors) {}
6
7 public record CloudflareImageResult(String image) {}
```

REST 호출과 응답 변환

```
1  CloudflareImageResponse response = restClient.post()
2      .uri("/accounts/" + props.accountId() + "/ai/run/" + model)
3      .body(new CloudflareImageRequest(prompt, steps, seed))
4      .retrieve()
5      .body(CloudflareImageResponse.class);
6
7  Image image = new Image(null, response.result().image());
8  return new ImageResponse(List.of(new ImageGeneration(image)));
```

호출부에서 걸려 넘어지는 지점

- 모델 ID 인코딩: @ · / 때문에 URI 변수로 넘기면 404 발생
- 200이어도 실패: 응답의 success와 이미지 존재 여부를 반드시 확인
- 접두사 위치: b64Json에는 순수 Base64만, Data URI는 화면에서 조립
- 타임아웃 필수: 무제한 대기는 요청 스레드를 고갈시킴

이미지 생성 서비스의 처리 흐름



생성 서비스 핵심부

```
1 String usedPrompt = enhance ? promptEnhancer.enhance(rawPrompt) : rawPrompt;
2 long seed = ThreadLocalRandom.current().nextLong(0, 4_294_967_296L);
3 CloudflareImageOptions options = CloudflareImageOptions.builder()
4     .seed(seed)
5     .build();
6
7 ImageResponse response = imageModel.call(new ImagePrompt(usedPrompt, options));
8 String base64 = response.getResult().getOutput().getB64Json();
```

선택했을 때만 저장하기

```
1 String storageKey = null;
2 if (save) {
3     byte[] bytes = Base64.getDecoder().decode(base64);
4     ImageMetadata metadata = new ImageMetadata(
5         rawPrompt, usedPrompt, props.model(), props.steps(), seed);
6
7     storageKey = generatedImageStorage.store(bytes, metadata);
8 }
```

폼 검증과 컨트롤러 연결

```
1  @PostMapping("/images/generate")
2  public String generate(@Valid @ModelAttribute ImageForm form,
3                          BindingResult bindingResult, Model model) {
4      if (bindingResult.hasErrors()) {
5          return "images/form";
6      }
7      model.addAttribute("generated", imageService.generate(form));
8      return "images/form";
9  }
```

결과 화면

```
1 <div th:if="${generated}" th:object="${generated}">
2   
3   <p th:text="*{usedPrompt}"></p>
4   <span th:text="*{seed}"></span>
5   <p th:if="*{storageKey}" th:text="${'저장 완료: ' + *{storageKey}}"></p>
6 </div>
```

화면에서 조심할 것

- 버튼 비활성화: 중복 제출만 막는 편의 기능, 사용량 제한이 아님
- 비동기 분리: 실무에서는 작업을 분리하고 상태 폴링을 검토
- Base64 보관: 여러 요청에 들고 다니지 않음(약 33% 증가)
- 예외 처리: 전역 예외 처리(`@ControllerAdvice`)로 사용자 메시지 변환

결과를 다루는 두 갈래

방식	장점	단점
Data URI 즉시 표시	저장 없이 바로 렌더링, 단순	캐시 불가, 새로고침 시 소멸
객체 스토리지 저장	캐시·재사용·이력 관리 가능	저장 비용과 정리 정책 필요

저장 시 함께 남길 메타데이터

필드	내용
<code>prompt</code> / <code>enhancedPrompt</code>	원본 프롬프트 / 보강된 프롬프트
<code>model</code> / <code>steps</code>	사용한 모델 ID / 디노이징 스텝 수
<code>seed</code> / <code>createdAt</code>	재현용 시드 / 생성 시각

- **재현성:** seed와 프롬프트를 함께 저장해야 결과를 재현 가능

운영에서 놓치기 쉬운 것

- 자격 증명: 실행 권한만 가진 전용 토큰을 서버에서만 사용
- 타임아웃: 지연이 길어 명시적 타임아웃과 재시도 안내 필요
- 콘텐츠 정책: 저작권·부적절 콘텐츠는 라이선스 및 Security 정책에서
- 로깅 주의: 프롬프트는 신중히, Base64 페이로드는 절대 출력 금지

학습 요약 (1/2)

- 확산 모델: 노이즈에서 출발해 잠재 공간에서 디노이징을 반복
- 핵심 파라미터: `steps` · `guidance` · `seed` 가 품질·재현성을 결정
- Workers AI: GPU 인프라 없이 REST 한 번으로 모델 호출
- FLUX.1 schnell: 4~8 스텝 증류 모델이라 REST 실습에 적합

학습 요약 (2/2)

- 프롬프트 설계: 명령문이 아닌 장면 묘사, LLM으로 영어 보강
- 커스텀 ImageModel: 호환 계층에 이미지 생성이 없어 직접 구현
- 결과 처리: Data URI 즉시 표시와 선택적 스토리지 저장을 조합
- 재현성: 원본 프롬프트와 seed를 함께 저장해야 결과 재현 가능